

Member 1 — Authentication & User Management

Main responsibility

Login, users and role-based access

UI

Create:

- Login Form
- User Management Form
- Add User Form
- Edit User Form
- Change Password Form

C# / OOP

Create classes such as:

User

Role

AuthenticationService

UserService

Implement:

- Login validation
- Logout
- Password handling
- Role checking
- User CRUD

Database

Main tables:

Users

Roles

Also interacts with:

Doctors

because doctor accounts are connected to users.

Features

- Login
- Logout
- Add user
- Edit user
- Activate/deactivate user
- Assign role
- Role-based menu access

Testing

Test:

- Correct login
- Wrong password
- Inactive account
- Wrong username
- Different role access

Member 2 — Patient Management

Main responsibility

Complete patient management

UI

Create:

- Patient List
- Add Patient

- Edit Patient
- Patient Details
- Patient Search

C# / OOP

Classes:

Patient

PatientService

PatientRepository

Implement:

- Add
- Update
- Search
- View
- Deactivate

Database

Main table:

Patients

Will also interact with:

Appointments

MedicalRecords

Prescriptions

Bills

Features

- Patient registration
- Generate patient code
- Search by patient code

- Search by NIC
- Search by name
- View patient history
- Update patient details

Testing

Test:

- Duplicate NIC
 - Missing required fields
 - Invalid phone number
 - Patient search
 - Patient update
-

Member 3 — Doctor & Department Management

Main responsibility

Doctor and hospital department management

UI

Create:

- Department Management
- Doctor Management
- Add Doctor
- Edit Doctor
- Doctor List

C# / OOP

Classes:

Doctor

Department

DoctorService

DepartmentService

Database

Main tables:

Doctors

Departments

Also:

Users

because every doctor has a login account.

Features

- Add department
- Edit department
- Add doctor
- Edit doctor
- Assign doctor to department
- Assign specialization
- Activate/deactivate doctor
- View doctors by department

Testing

Test:

- Duplicate doctor code
 - Invalid department
 - Doctor account relationship
 - Doctor search
 - Department filtering
-

Member 4 — Appointment Management

Main responsibility

Complete appointment system

UI

Create:

- Appointment Dashboard
- Book Appointment
- Edit Appointment
- Cancel Appointment
- Appointment Search
- Doctor Schedule

C# / OOP

Classes:

Appointment

AppointmentService

AppointmentRepository

Database

Main table:

Appointments

Uses:

Patients

Doctors

Users

Features

- Book appointment
- Select patient

- Select doctor
- Select date
- Select time
- Appointment status
- Cancel appointment
- Reschedule appointment
- View doctor schedule

Important business logic

Check:

Does this doctor already have an appointment during this time?

If yes:

Appointment cannot be booked.

Testing

Test:

- Overlapping appointments
- Invalid time
- Cancelled appointment
- Rescheduling
- Different doctors

Member 5 — Medical Records & Medical History

Main responsibility

Doctor consultation and patient medical history

UI

Create:

- Medical Record Form
- Consultation Form
- Medical History View
- Patient History Search

C# / OOP

Classes:

MedicalRecord

MedicalRecordService

MedicalRecordRepository

Database

Main table:

MedicalRecords

Uses:

Patients

Doctors

Appointments

Features

Doctor can record:

Symptoms

Diagnosis

Treatment

Notes

Visit Date

Patient history should display:

Previous visits

Previous diagnosis

Previous treatment

Previous doctor

Testing

Test:

- Creating medical record
 - Linking correct patient
 - Linking correct doctor
 - Viewing history
 - Appointment relationship
-

Member 6 — Prescription & Pharmacy

This is one of the larger modules, so give it to one member who is comfortable with C#.

Main responsibility

Prescription + medicine + pharmacy operations

UI

Create:

- Medicine Management
- Medicine Search
- Prescription Form
- Prescription Details
- Pharmacy Dashboard
- Dispense Prescription

C# / OOP

Classes:

Medicine

Prescription

PrescriptionItem

PharmacyService

PrescriptionService

Database

Main tables:

Medicines

Prescriptions

PrescriptionItems

Uses:

Patients

Doctors

MedicalRecords

Features

Medicine

- Add medicine
- Edit medicine
- Search medicine
- View stock
- Low-stock warning

Prescription

- Create prescription
- Add multiple medicines
- Dosage
- Frequency
- Duration
- Instructions

Pharmacy

- View prescription
- Dispense medicine
- Reduce stock
- Mark prescription as dispensed

Important business rule

If:

Requested Quantity > Stock Quantity

then:

Cannot dispense.

Testing

Test:

- Insufficient stock
- Multiple medicines
- Stock reduction
- Prescription status
- Medicine search

Member 7 — Billing & Payments

Main responsibility

Hospital billing and payment management

UI

Create:

- Billing Form
- Bill Search
- Payment Form

- Payment History
- Outstanding Bills

C# / OOP

Classes:

Bill

Payment

BillingService

PaymentService

Database

Main tables:

Bills

Payments

Uses:

Patients

Appointments

Users

Features

- Create bill
- Calculate subtotal
- Apply discount
- Calculate total
- Record payment
- View payment history
- Calculate outstanding balance
- Mark bill as paid

Example

Bill = Rs. 10,000

Payment = Rs. 4,000

Remaining = Rs. 6,000

Testing

Test:

- Full payment
 - Partial payment
 - Multiple payments
 - Invalid payment amount
 - Payment greater than balance
-

Member 8 — Dashboard, Reports & System-Wide Search

This member should **not just make a pretty dashboard**.

They should build the system's reporting functionality.

UI

Create:

- Main Dashboard
- Reports Form
- Search/Filter screens
- Summary screens

C# / OOP

Classes:

DashboardService

ReportService

ReportRepository

Database

This member mainly performs **read-only queries across multiple tables**.

For example:

Patients

Appointments

Doctors

MedicalRecords

Medicines

Bills

Payments

Dashboard

Display things such as:

Total Patients

Today's Appointments

Completed Appointments

Available Doctors

Low Stock Medicines

Pending Bills

Today's Payments

Reports

Create realistic reports:

1. Patient List Report
2. Appointment Report
3. Doctor Appointment Report
4. Patient Medical History

5. Medicine Stock Report

6. Low Stock Report

7. Prescription Report

8. Billing Report

9. Payment Report

Testing

Test:

- Date filtering
- Doctor filtering
- Patient filtering
- Empty results
- Correct totals

Member 9 — Database Integration + System Integration

This person should **also have a proper functional module**, rather than becoming someone who only waits until the end.

I recommend giving them:

System Administration & Integration

UI

Create:

- Admin Dashboard
- System Settings/basic configuration
- Database status/test screen if appropriate
- Integration-related screens

They can also assist with common UI components.

C# / OOP

Classes:

DatabaseConnection

BaseRepository

ValidationHelper

ExceptionHandler

SystemService

And integration support for:

Patient

Appointment

Doctor

Billing

Pharmacy

Database

Responsible for maintaining the **master database**:

Roles

Users

Departments

Doctors

Patients

Appointments

MedicalRecords

Medicines

Prescriptions

PrescriptionItems

Bills

Payments

Main responsibility

This member ensures:

Member 1



Member 2



Member 3



...

Member 8



ONE DATABASE



ONE APPLICATION

Testing

Responsible for:

- Integration testing
- Foreign-key problems
- Database connection
- Common SQL errors
- Merge conflicts
- Final system testing

Very Important: Member 9 should NOT own everyone's work

This is important.

Don't make Member 9:

"The person who does all integration."

Instead:

Each member

↓

Builds own module

↓

Tests own module

↓

Commits code

↓

Member 9 integrates

↓

Member + Member 9 test together

So Member 9 is the **integration coordinator**, not the person who fixes everyone's unfinished work.

Complete 9-Member Allocation

Here is the clean version you can put directly into your workload document:

Member	Module	UI	C# / OOP	Database	Testing
1	Authentication & Users	Login/User Forms	User, Role, Auth	Users, Roles	Login & security
2	Patient Management	Patient Forms	Patient classes	Patients	Patient CRUD
3	Doctor & Departments	Doctor/Dept Forms	Doctor, Department	Doctors, Departments	Doctor/Dept
4	Appointments	Appointment Forms	Appointment classes	Appointments	Scheduling
5	Medical Records	Consultation/History	MedicalRecord classes	MedicalRecords	Medical history
6	Prescription & Pharmacy	Prescription/Pharmacy	Medicine, Prescription	3 pharmacy tables	Stock/prescription
7	Billing & Payments	Billing/Payment	Bill, Payment	Bills, Payments	Payment/billing
8	Dashboard & Reports	Dashboard/Reports	Report classes	Multi-table queries	Reports
9	System Admin + Integration	Admin/Common UI	Integration/DB classes	Master schema	Integration

Standard folder structure for EVERY member

I strongly recommend everyone follows the same project structure.

MediCareHospital

|

|— Forms

|

|— Models

|

|— Services

|

|— Repositories

|

|— Helpers

|

└ Database

For example, Member 2:

Forms

|— PatientForm.cs

|— AddPatientForm.cs

└ EditPatientForm.cs

Models

└ Patient.cs

Services

└ PatientService.cs

Repositories

└─ PatientRepository.cs

Member 4:

Forms

└─ AppointmentForm.cs

└─ AppointmentListForm.cs

Models

└─ Appointment.cs

Services

└─ AppointmentService.cs

Repositories

└─ AppointmentRepository.cs

This consistency will make your final integration **much easier**.

Workflow for every each one

